Langages et algorithmique	Chap 05	Calculabilité et décidabilité

5.1 Programme, problème et décision

Cela devient meta

Depuis le début de l'année, nous écrivons des programmes qui résolvent des problèmes.

- trier une liste,
- rechercher un élément,
- calculer un plus court chemin.

Nous avons alors toujours exhibé un programme/algorithme qui résolvait ces problèmes mais..

Existe-t-il des problèmes qu'aucun programme ne pourra jamais résoudre ?

Pour répondre à cette question, il faut d'abord comprendre précisément ce qu'est un **programme** et ce qu'est un **problème**.

5.1.1 Un programme est une donnée

C'est que des lettres hein !

Certes un programme est une liste d'instructions qui permet l'exécution d'un travail sur une machine, **CEPENDANT :**
Un programme informatique n'est rien d'autre qu'une **suite finie de symboles** écrits dans un langage formel. Autrement dit :

Un programme est un texte.

Ce texte peut être :

- enregistré dans un fichier,
- copié ou modifié,
- manipulé par un autre programme.

Remarque 1 (Ce n'est pas anodin)**.** Si un programme est une donnée, alors un programme peut prendre **un autre programme en entrée**.

C'est exactement ce que font des logiciels comme les interpréteurs, les compilateurs, les analyseurs de code, ou même les antivirus. On peut même se demander

" Peut-on écrire un programme capable de dire, pour n'importe quel programme, s'il va s'arrêter ou non ? "

Exemple 1 — Un interpréteur très simplifié

On peut écrire une fonction capable d'exécuter un programme donné sous forme de chaîne de caractères :

```
def universel(algo, *args):
    exec(algo)
    ligne1 = algo.split('\n')[0]
    nom = ligne1.split('(')[0][4:]
    return eval(f"{nom}{args}")
```

Ici :

- `algo` est une chaîne de caractères ;
- cette chaîne contient la définition d'une fonction ;
- on exécute ce texte avec `exec` ;
- puis on appelle la fonction définie.

Par exemple :

```
programme = "def carre(x):\n\treturn x*x"

print(universel(programme, 5))
```

Ce code affiche :

$$25$$

Un programme est donc ici traité comme une simple donnée.

5.1.2 Notion de problème

Avant d'aller plus loin, il faut clarifier ce que l'on appelle un **problème**.

Définition 1 — Problème

Un **problème** est une question générale portant sur un ensemble (souvent infini) d'entrées possibles.
Une **instance** est une entrée particulière du problème.
Une **solution** est une réponse correcte pour une instance donnée.

Exemple 2

Problème : déterminer si un entier est premier.
Instance : 17.
Solution : oui.

Instance : 21.
Solution : non.

Lorsque nous écrivons un programme, nous cherchons en réalité à construire une procédure générale capable de résoudre toutes les instances d'un problème.

5.1.3 Les problèmes de décision

Définition 2 — Problème de décision

Un **problème de décision** est un problème dont la réponse est OUI ou NON.
On peut le modéliser par une fonction :

$$f : \Sigma^* \to \{0, 1\}$$

où :

- Σ est un alphabet fini ;
- Σ^* est l'ensemble des mots finis sur Σ ;
- 1 représente OUI et 0 représente NON.

Autrement dit, une fonction qui associe à toute entrée la valeur OUI ou NON.

Propriété 1 (Deux formulations équivalentes)**.** Un problème de décision peut être vu soit comme une question dont la réponse est OUI ou NON, soit comme une fonction

$$f : \Sigma^* \to \{0, 1\}$$

qui associe à chaque entrée 1 (oui) ou 0 (non).

Poser la question " x vérifie-t-elle la propriété ? " revient exactement à définir la fonction $f(x)$.

Transformer un problème en problème de décision

En informatique théorique, presque tous les problèmes sont étudiés sous forme de problèmes de décision, car cela permet un cadre mathématique plus clair.
Pour passer d'un problème à un problème de décision on remplace une recherche optimale par une question du type

" Existe-t-il une solution satisfaisant telle contrainte ? "

Exemple 3 — Transformer en problème de décision

Considérons le problème suivant :

Problème : trouver le plus court chemin entre deux sommets d'un graphe.

Ce problème ne renvoie pas OUI ou NON : il renvoie un chemin..

Pour en faire un problème de décision, on introduit un paramètre k :

Problème de décision associé :
" Existe-t-il un chemin de longueur $\leq k$ entre ces deux sommets ? "

La réponse est maintenant OUI ou NON.

Exercice 1 — Transformer en problème de décision

Transformer les problèmes suivants en problèmes de décision :

1. Trier une liste.

   ---

2. Trouver un cycle eulérien dans un graphe.

   ---

3. Résoudre un Sudoku.

5.1.4 Algorithme et décidabilité

Nous pouvons maintenant formaliser ce que signifie " résoudre un problème ".

Définition 3 — Décideur

Soit $f : \Sigma^* \to \{0, 1\}$ un problème de décision.
Un **décideur** pour f est un algorithme qui, pour toute entrée x :

— termine ;

— renvoie la valeur correcte $f(x)$.

Définition 4 — Problème décidable

Un problème est **décidable** s'il existe un décideur pour ce problème.
Sinon, le problème est dit **indécidable**.

Exercice 2 — La condition essentielle

On considère le programme suivant :

```
def A(x):
    while x>10:
        x=x+1
```

1. Ce programme termine-t-il forcément ?

   ----

2. Peut-il être un décideur ?

5.2 Le problème de l'arrêt

En voilà une bonne question

Nous savons maintenant définir ce qu'est un décideur.
Posons alors la question suivante :

Existe-t-il un programme capable de décider, pour tout programme P et toute entrée x, si $P(x)$ termine ou non ?

Autrement dit :

Peut-on écrire un programme qui analyse n'importe quel programme et détermine s'il s'arrête ?

Ce problème s'appelle le **problème de l'arrêt**.

Définition 5 — Problème de l'arrêt

Entrée :

- un programme P ;
- une entrée x.

Question :

Le programme P, exécuté avec l'entrée x, termine-t-il ?

Théorème 1 — Problème de l'arrêt

Le problème de l'arrêt est **indécidable**.

5.2.1 Démonstration par l'absurde

Supposons qu'il existe un décideur que nous appellerons `Arret` :

```
Arret(P, x)
```

qui renvoie :

- 1 si $P(x)$ termine ;
- 0 sinon.

Et supposons qu'il fonctionne pour **tout** programme P et toute entrée x.
Construisons alors le programme suivant :

```python
def Paradoxe(P):
    if Arret(P, P) == 1:
        while True:
            pass
    else:
        return 0
```

Exercice 3 — Analyse du programme

On suppose que `Arret` fonctionne parfaitement.
Que se passe-t-il si l'on exécute :

```
Paradoxe(Paradoxe)
```

1. Si `Arret(Paradoxe, Paradoxe)` renvoie 1, que fait `Paradoxe(Paradoxe)` ?

   ---

2. Si `Arret(Paradoxe, Paradoxe)` renvoie 0, que fait `Paradoxe(Paradoxe)` ?

   ---

3. En quoi cela fait-il émerger des contradictions ?

   ---

4. Que peut-on conclure sur le problème de l'arrêt ?

5.3 Modéliser la notion d'algorithme

Pourquoi formaliser ?

Dans la section précédente, nous avons montré qu'il existe un problème indécidable.
Mais nous avons raisonné en parlant de " programme " ou d' "algorithme" de manière informelle.
Pour être pleinement rigoureux, il faut répondre à la question :

Qu'appelle-t-on exactement un algorithme ?

5.3.1 La machine de Turing

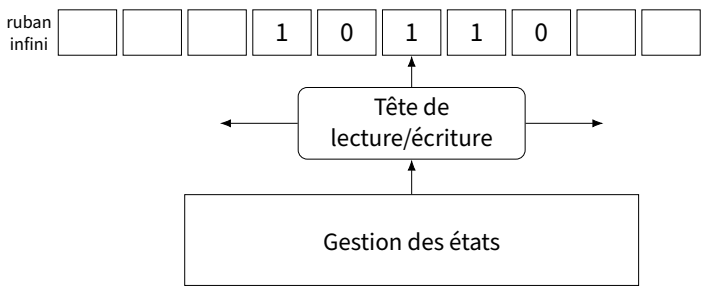
En 1936, Alan Turing propose un modèle abstrait de calcul extrêmement simple, appelé **machine de Turing** pour décrire tout calcul mécanique.

Idée générale

Une machine de Turing est constituée :

- d'un ruban infini découpé en cases,
- d'une tête de lecture/écriture,
- d'un nombre fini d'états,
- d'une table d'instructions.

À chaque étape, la machine commence par lire le symbole courant au niveau de sa tête de lecture, puis elle écrit éventuellement un symbole. Ensuite la tête se déplace à gauche ou à droite et enfin la machine change d'état.
L'entrée d'une machine de Turing correspond à la disposition du ruban au début de l'éxécution. La tête de lecture se place au début de l'entrée et toutes les autres cases du ruban contiennent un symbole blanc (⊔).
La sortie d'une machine de Turing correspond à la disposition du ruban lorsque l'éxécution est terminée.



ruban infini

État	Lit	Écrit	Déplace	État suivant
q_0	⊔	⊔	D	q_0
	1	1	D	q_1
	0	0	D	q_1
q_1	⊔	⊔	D	q_f
	1	1	D	q_1
	0	1	D	q_1
q_f	État final (arrêt)			

FIGURE 1 – Représentation d'une machine de Turing

Définition 6 — Machine de Turing

Une machine de Turing est définie par un tuple

$$(Q, \Sigma, \Gamma, \delta, q_0, q_f)$$

où :

- Q est un ensemble fini d'états ;
- Σ est l'alphabet d'entrée ;
- Γ est l'alphabet du ruban ($\Sigma \subseteq \Gamma$) ;
- δ est la fonction de transition ;
- q_0 est l'état initial.
- q_0 est l'état final.

La fonction δ indique, pour chaque état et symbole lu, le nouvel état, le symbole à écrire et le déplacement de la tête.

Remarque 2. Une machine de Turing peut être vue comme une formalisation mathématique de ce que fait un programme.

Du ruban à l'automate

Il est possible de représenter la fonction de transition d'une machine de Turing sous forme de diagramme dans lequel

- chaque état est représenté par un cercle ;
- les transitions sont des flèches ;
- chaque flèche est étiquetée par un triplet :

lecture ; écriture ; déplacement.

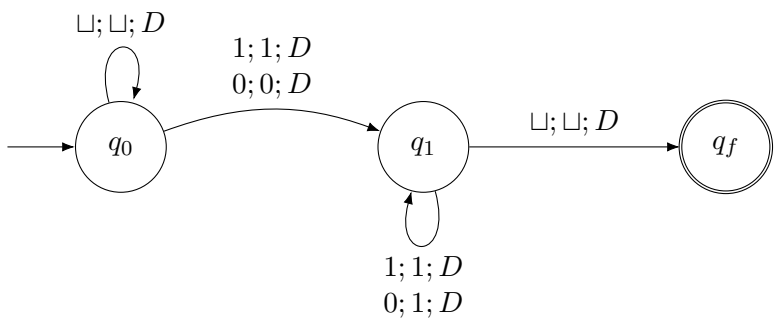


FIGURE 2 – Diagramme d'automate correspondant à la fonction de transition.

Exercice 4 — Inverser les bits avec une machine de Turing

On souhaite écrire une machine de Turing qui, sur une entrée binaire, **remplace chaque** 0 **par** 1 **et chaque** 1 **par** 0, puis s'arrête.

1. A partir du diagramme d'automate suivant, établir la **table de transition** (c'est-à-dire remplir les colonnes *État, Lit, Écrit, Déplace, Suivant*).

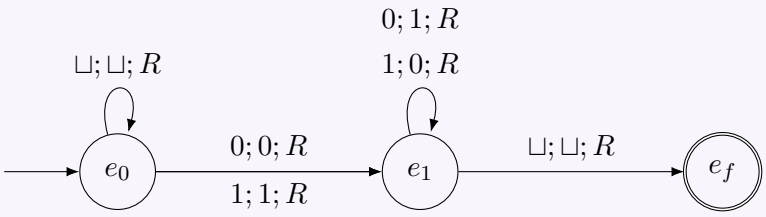


Table à compléter :

État	Lit	Écrit	Déplace	Suivant
e_0	$\sqcup$			
e_0	0			
e_0	1			
e_1	0			
e_1	1			
e_1	$\sqcup$			

2. Suivre pas à pas l'évolution du ruban si le ruban d'éntrée vaut `10110` ?

5.3.2 Calculabilité au sens de Turing

Nous pouvons désormais être précis lorsque nous parlons de calculabilité.

Définition 7 — Nombre calculable

Un nombre réel x est dit **calculable** s'il existe une machine de Turing qui, pour tout entier n, peut produire une approximation de x à n décimales près.
Autrement dit, il existe un algorithme capable de calculer les décimales de x avec une précision arbitraire.

Ça fait réfléchir..

On peut donc se demander si tous les nombres sont calculables ?

Définition 8 — Fonction calculable

Soit $f : \Sigma^* \to \Sigma^*$.
On dit que f est **calculable** s'il existe une machine de Turing M telle que, pour toute entrée x :

- la machine s'arrête ;
- le contenu du ruban final représente $f(x)$.

Autrement dit, une fonction est calculable si elle peut être réalisée par une machine de Turing.

Définition 9 — Décideur

Soit $f : \Sigma^* \to \{0, 1\}$ un problème de décision.
Un **décideur** pour f est une **machine de Turing** qui, pour toute entrée x :

- termine ;
- renvoie la valeur correcte $f(x)$.

Définition 10 — Problème décidable

Un problème est **décidable** s'il existe un décideur pour ce problème.
Sinon, le problème est dit **indécidable**.

5.3.3 Universalité

Machine universelle

Turing montre qu'il existe une machine particulière, appelée **machine universelle**, capable de simuler toute autre machine de Turing.
Autrement dit :

Il existe une machine qui peut exécuter la description d'une autre machine.

C'est l'équivalent théorique de nos ordinateurs modernes.

5.3.4 D'autres modèles équivalents

Pas que le modèle de Turing.. mais en fait si ?

La machine de Turing n'est pas le seul modèle de calcul proposé.
À la même époque :

- Alonzo Church introduit le λ-calcul ;
- Gödel et Kleene développent la théorie des fonctions récursives.

Il en existe un tas d'autres :

- machines RAM (modèle de nos ordinateurs, Von Neumann)
- machines à compteurs (Marvin Minsky)
- les automates cellulaires (ex. : jeu de la vie de Conway)
- la redstone dans Minecraft
- tous les langages de programmation

Ces modèles sont très différents en apparence, mais un fait remarquable apparaît :

Ils décrivent exactement les mêmes fonctions calculables.

5.3.5 La thèse de Church-Turing

Thèse de Church-Turing

Tout calcul qui peut être effectué par un procédé mécanique peut être réalisé par une machine de Turing.
Autrement dit :

La notion intuitive d'algorithme coïncide avec la notion de calculabilité au sens des machines de Turing.

Remarque 3 (C'est une thèse)**.** La thèse de Church-Turing n'est pas un théorème, car elle relie une notion informelle (" algorithme ") à une notion formelle (" calculabilité par machine de Turing ").

5.4 Complexité

Décidable ne veut pas dire décidable rapidement

Nous avons distingué :

— les problèmes décidables ;

— les problèmes indécidables.

Mais parmi les problèmes décidables, tous ne sont pas équivalents.
Certains peuvent être résolus rapidement, d'autres nécessitent un temps de calcul gigantesque.
La nouvelle question devient alors :

Un problème décidable peut-il être résolu efficacement ?

C'est l'objet de la théorie de la complexité.

5.4.1 Temps de calcul

Définition 11 — Temps de calcul

Le **temps de calcul** d'une machine de Turing est le nombre d'étapes effectuées avant l'arrêt de la machine, en fonction de la taille de l'entrée.
On note généralement cette taille par n.

Remarque 4. On ne s'intéresse pas au temps exact, mais à la manière dont il évolue lorsque n grandit.
Par exemple :

$$n, \quad n^2, \quad n^3$$

restent raisonnables lorsque n augmente,
alors que :

$$2^n, \quad 3^n$$

deviennent rapidement gigantesques.

5.4.2 La classe P

Définition 12 — Classe P

Un problème de décision appartient à la classe **P** s'il existe une machine de Turing qui le décide en un temps polynomial en la taille de l'entrée :

$$O(n^k)$$

pour un certain entier k.

Exemple 4

Exemples de problèmes appartenant à la classe P :

- déterminer si un nombre est pair ;
- trier une liste ;
- calculer un plus court chemin.

Ces problèmes sont considérés comme efficacement résolubles.

5.4.3 La classe NP

Définition 13 — Classe NP

Un problème de décision appartient à la classe **NP** s'il existe une machine de Turing qui peut vérifier en temps polynomial qu'une solution proposée est correcte.

Remarque 5. **NP** ne veut **PAS** dire "Non-polynomial" mais *Non-deterministic Polynomial time*

Résoudre vs vérifier

Il peut être difficile de trouver une solution, mais facile de vérifier qu'une solution est correcte. Par exemple :

- Trouver une solution d'une grille de soduku est difficle.
- Vérifier qu'une grille donnée est correcte ou non est rapide.

La classe NP regroupe les problèmes pour lesquels la vérification est efficace.

5.4.4 Relations entre P et NP

Propriété 2. Tout problème de la classe P appartient à la classe NP.

En effet, si l'on peut résoudre un problème rapidement, on peut évidemment vérifier rapidement une solution.

On a donc :

$$\mathbf{P} \subseteq \mathbf{NP}$$

Le problème P = NP ?

La grande question ouverte en informatique théorique est :

P = NP ?

Autrement dit :
Tout problème dont on peut vérifier rapidement une solution peut-il être résolu rapidement ?

À ce jour, personne ne connaît la réponse et 1 million d'euros est offert par *The Clay Mathematics Institute of Cambridge* pour celui ou celle qui saura y répondre !

Remarque 6. Si l'on découvrait que $P = NP$, de nombreux problèmes considérés comme difficiles deviendraient efficacement résolubles.

5.5 Culture et références

Des machines à calculer aux ordinateurs

Wilhelm Schickard (1592–1635), professeur à l'Université de Tübingen, aurait dessiné les plans de la première machine à calculer mécanique. Elle ne fut jamais construite, mais l'idée d'automatiser le calcul était déjà présente.

Blaise Pascal (1623–1662), à seulement 19 ans, construit la *Pascaline*, première machine à calculer opérationnelle. Elle permet d'effectuer des additions afin d'aider son père dans ses calculs fiscaux.

Gottfried Wilhelm Leibniz (1646–1716) améliore ces machines et développe une machine capable d'effectuer les quatre opérations. Il défend des idées très modernes :

— utilisation de la base 2 ;

— schéma entrée–calcul–sortie ;

— rêve d'une " machine de calcul universelle ".

Le métier à tisser de **Joseph Marie Jacquard** (1752–1834), fonctionnant avec des cartes perforées, introduit l'idée qu'une machine peut être pilotée par un programme matériel.

Charles Babbage (1791–1871), professeur à Cambridge, conçoit la machine différentielle, puis imagine la machine analytique. Cette dernière contient déjà :

— une unité de calcul,

— une mémoire,

— un dispositif de contrôle,

— un système d'entrée-sortie.

Elle peut être considérée comme le véritable ancêtre conceptuel de l'ordinateur moderne.

Ada Lovelace : première programmeuse

Ada Lovelace (1815–1852) travaille avec Charles Babbage sur la machine analytique.
Dans ses notes, elle rédige ce qui est considéré comme le premier programme informatique de l'histoire, destiné à être exécuté par une machine.
Mais son apport dépasse largement cela :

— Elle comprend que ces machines ne se limitent pas au calcul numérique.

— Elle imagine qu'elles pourraient manipuler des symboles, de la musique, voire toute forme d'information.

Elle entrevoit donc l'idée d'ordinateur universel, bien avant sa réalisation concrète.
Le langage de programmation `Ada` a été nommé en son honneur.

Logique, décidabilité et fondements

En 1900, **David Hilbert** propose 23 problèmes majeurs. Il lance ensuite le programme de Hilbert, visant à formaliser entièrement les mathématiques.
Il pose notamment l'*Entscheidungsproblem* : existe-t-il une méthode mécanique permettant de décider si un énoncé mathématique est vrai ou faux ?

En 1931, **Kurt Gödel** montre qu'un système formel suffisamment puissant est nécessairement incomplet ou incohérent. Il construit une formule qui affirme sa propre indémontrabilité.

En 1936, **Alan Turing** et **Alonzo Church** montrent indépendamment que l'Entscheidungsproblem est indécidable.
Turing introduit la machine de Turing, Church le λ-calcul.
Ils formulent ce que l'on appellera la **thèse de Church-Turing** : toute procédure de calcul mécanique peut être simulée par ces modèles.

Pour aller plus loin

Vidéo pédagogique :
Preuve du problème de l'arrêt - Vincent Guirardel : https://www.youtube.com/watch?v=a5MNIzu9Ia4
Preuve du problème de l'arrêt : https://www.youtube.com/watch?v=92WHN-pAFCs
Les machines de Turing https://www.youtube.com/watch?v=L5O04P2ASRc
La machine de Turing, film de Florence RIOU https://www.youtube.com/watch?v=L5O04P2ASRc

Cours universitaire : François Schwartzentruber *Kit de survie – Calculabilité* https://cel.hal.science/cel-01947220